Web Application Security Assessment

PortSwigger Web Security Academy — Complete Penetration Testing Report

Target: CBA Portfolio Task 4
Status: 11/11 Resolved
Scope: Full Assessment

1. EXECUTIVE SUMMARY

During this technical assessment, eleven (11) distinct web application vulnerabilities across four core threat vectors —Cross-Site Scripting (XSS), SQL Injection (SQLi), Path Traversal, and Information Disclosure—were identified and successfully exploited. All security defects were thoroughly validated without relying on external proxies or command-line scripting, demonstrating direct web-client exploitability.

11	5	4	2
TOTAL VULNERABILITIES	HIGH SEVERITY	MEDIUM SEVERITY	LOW SEVERITY

2. VULNERABILITY ASSESSMENT MATRIX

#	Vulnerability Category	Lab Name	Severity	Status
01	Cross-Site Scripting	Reflected XSS into HTML context with nothing encoded	HIGH	✅ Solved
02	Cross-Site Scripting	Stored XSS into HTML context with nothing encoded	HIGH	✅ Solved
03	Cross-Site Scripting	DOM XSS in `document.write` sink using `location.search`	MEDIUM	✅ Solved
04	Cross-Site Scripting	DOM XSS in jQuery anchor `href` attribute sink	MEDIUM	✅ Solved
05	Cross-Site Scripting	Reflected XSS into attribute with angle brackets encoded	HIGH	✅ Solved
06	Cross-Site Scripting	Stored XSS into anchor `href` attribute	HIGH	✅ Solved
07	SQL Injection	SQL injection vulnerability allowing login bypass	HIGH	✅ Solved
08	SQL Injection	SQL injection in WHERE clause allowing retrieval of hidden data	MEDIUM	✅ Solved
09	SQL Injection	SQL injection UNION attack (determining column count)	MEDIUM	✅ Solved
10	Path Traversal	File path traversal, simple case	HIGH	✅ Solved
11	Information Disclosure	Information disclosure in error messages	LOW	✅ Solved

3. DETAILED TECHNICAL FINDINGS (1 - 4)

01. Reflected XSS into HTML Context

HIGH SEVERITY

PAYLOAD / VECTOR
`<script>alert(1)</script>` (Search Input)

IMPACT
Arbitrary JavaScript execution in user session via unescaped search string reflecting directly into DOM.

02. Stored XSS into HTML Context

HIGH SEVERITY

PAYLOAD / VECTOR
`<script>alert(1)</script>` (Comment Body)

IMPACT
Persistent script execution affecting all users viewing the blog post. Can lead to session hijacking.

03. DOM XSS in document.write Sink

MEDIUM SEVERITY

PAYLOAD / VECTOR
`"><script>alert(1)</script>` (URL Search query)

IMPACT
Client-side code dynamically writes untrusted URL parameters into the DOM via `document.write`.

04. DOM XSS in jQuery Anchor href Sink

MEDIUM SEVERITY

PAYLOAD / VECTOR
`javascript:alert(1)` in `returnPath` parameter

IMPACT
Client-side jQuery dynamically sets back-link `href` attributes from query strings without scheme validation.

05. Reflected XSS into Attribute (Encoded Brackets)

HIGH SEVERITY

PAYLOAD / VECTOR
`" autofocus onfocus=alert(1) x="`

IMPACT
Bypasses angle-bracket filtering by breaking out of HTML attribute value using quotes and event handlers.

3. DETAILED TECHNICAL FINDINGS (6 - 11)

06. Stored XSS in Anchor href Attribute

HIGH SEVERITY

PAYLOAD / VECTOR
`javascript:alert(1)` (Website Form Field)

IMPACT
Comment author links store `javascript:` pseudo-protocols, executing code when clicked.

07. SQL Injection Login Bypass

HIGH SEVERITY

PAYLOAD / VECTOR
`administrator'--` (Username Field)

IMPACT
Completely bypasses authentication controls, granting administrative console access without credentials.

08 & 09. SQLi WHERE Clause & UNION Exploration

MEDIUM SEVERITY

PAYLOADS
`' OR 1=1--` | `' UNION SELECT NULL, NULL, NULL--`

IMPACT
Exposes unreleased products and probes underlying SQL schema query column alignment for data extraction.

10 & 11. Path Traversal & Information Disclosure

HIGH SEVERITY

PAYLOADS
`../../../etc/passwd` | String in `productId`

IMPACT
Reads arbitrary system files from server root and triggers unhandled stack traces revealing application framework versions.

4. STRATEGIC REMEDIATION ROADMAP

- **Context-Aware Context Encoding:** Implement robust HTML entity and attribute encoding across all user reflection points to mitigate XSS vulnerabilities.
- **Parameterized SQL Queries:** Enforce the use of prepared statements and parameterized queries (e.g., PDO, PreparedStatement) to eliminate all SQL injection risks.
- **Strict Directory Traversal Defense:** Validate file requests against an explicit whitelist and enforce absolute path resolution using `realpath()` functions.
- **Custom Error Handling:** Disable verbose stack traces in production environments; return standardized HTTP 500 error pages.